LeetCode 28: Find the Index of the First Occurrence in a String

1. Problem Title & Link

Title: LeetCode 28: Find the Index of the First Occurrence in a String

Link: https://leetcode.com/problems/find-the-index-of-the-first-occurrence-in-a-string/

2. Problem Statement (Short Summary)

Given two strings:

haystack

needle

Return the index of the **first occurrence** of needle inside haystack.

If needle does not exist:

return -1

Example

Input:

haystack = "sadbutsad"

needle = "sad"

Output:

0

Because:

sadbutsad

^^^

First occurrence starts at index:

0

3. Examples (Input → Output)

Example 1

Input:

haystack = "sadbutsad"

needle = "sad"

Output:

0

Example 2

Input:

haystack = "leetcode"

needle = "leeto"

Output:

-1

Example 3

Input:

haystack = "hello"

needle = "ll"

Output:

2

Example 4

Input:

haystack = "aaaaa"

needle = "bba"

Output:

-1

4. Constraints

1 <= haystack.length <= 10^4

1 <= needle.length <= 10^4

Important:

Need FIRST occurrence.

5. Core Concept (Pattern / Topic)

String Matching

Secondary Topics:

- Sliding Window
- Substring Comparison

6. Thought Process (Step-by-Step Explanation)

Brute Force Idea

Check every possible starting position.

Example:

haystack = hello

needle = ll

Length:

hello -> 5

ll -> 2

Possible starts:

0

1

2

3

Check:

he

No.

Check:

el

No.

Check:

ll

Match.

Return:

2

Key Observation

If:

needle length = m

Then last valid starting position is:

n - m

because after that there aren't enough characters left.

Example

haystack = hello

needle = ll

n = 5

m = 2

Last possible start:

5 - 2 = 3

7. Visual / Intuition Diagram (ASCII Diagram)

```
Input:
hello

ll
Check:
hello
^^

he
No.

hello
 ^^

el
No.

hello
  ^^

ll
Match.
```

Return:

2

8. Pseudocode (Language Independent)

```
for i from 0 to n-m

    if haystack[i:i+m] == needle

         return i

return -1
```

9. Code Implementation

Python

```
class Solution:
    def strStr(self, haystack: str, needle: str) -> int:

        n = len(haystack)
        m = len(needle)

        for i in range(n - m + 1):

            if haystack[i:i+m] == needle:
                return i

        return -1
```

Java

```
class Solution {

    public int strStr(String haystack,
                      String needle) {

        int n = haystack.length();
        int m = needle.length();

        for(int i = 0;
            i <= n - m;
            i++) {

            if(haystack.substring(i, i + m)
                    .equals(needle)) {

                return i;
            }
        }

        return -1;
    }
}
```

10. Time & Space Complexity

Time Complexity

$O((n-m+1) \times m)$

Worst case:

$O(nm)$

Space Complexity

O(1)

11. Common Mistakes / Edge Cases

Mistake 1

Looping till:

n

instead of:

n-m

Can cause index errors.

Mistake 2

Returning last occurrence.

Need:

First occurrence

Edge Case

Input:

haystack = "aaaaa"

needle = "aa"

Output:

0

Not:

1

2

3

12. Detailed Dry Run

Input:

haystack = "hello"

needle = "ll"

i	Substring	Match?
0	he	No
1	el	No

2	ll	Yes

Return:

2

13. Common Use Cases (Real-Life / Interview)

Text Search

Searching keywords inside documents.

Search Engines

Finding pattern occurrence.

DNA Matching

Finding gene sequences.

IDE Search

Ctrl + F functionality.

14. Common Traps (Important!)

Trap 1

Confusing substring with subsequence.

Need:

Continuous characters

Trap 2

Searching beyond valid range.

Trap 3

Not stopping after first match.

15. Builds To (Related LeetCode Problems)

LC 28 - Find First Occurrence

↓

LC 459 - Repeated Substring Pattern

↓

LC 438 - Find All Anagrams

↓

LC 567 - Permutation in String

↓

KMP Algorithm

16. Alternate Approaches + Comparison

Approach	Time	Space	Interview Rating
Brute Force	O(nm)	O(1)	Good
Built-in find()	O(n) avg	O(1)	Easy
KMP	O(n+m)	O(m)	Advanced

Approach 1: Brute Force (Chosen)

Most interview-friendly for Easy level.

Approach 2: Built-In

Python:

return haystack.find(needle)

Works but interviewer may ask manual implementation.

Approach 3: KMP

Advanced string matching.

Used for large pattern searches.

17. Why This Solution Works (Short Intuition)

Every possible starting position is checked exactly once.

If the substring beginning at that position matches the needle, that index is the first occurrence and is returned immediately.

18. Variations / Follow-Up Questions

Follow-Up 1

Return all occurrences.

Example:

aaaaa

aa

Output:

[0,1,2,3]

Follow-Up 2

Case-insensitive search.

Follow-Up 3

Implement KMP.

Follow-Up 4

Implement wildcard matching.

Interview Takeaway

Whenever you hear:

Find Pattern

Search String

Locate Substring

think:

String Matching

Sliding Window

KMP (Advanced)